University of Kansas

Algorithmic Algebraic Parser
Software Architecture Document

Version <1.6>

Revision History

Date	Version	Description	Author
<dd/mm/yy>	<x.x>	<details>	<name>
24/03/2026	1.0	Added content to 1.1, 1.3, 1.4, 1.5	Lincoln Earnshaw
31/03/2026	1.1	Retitled company name and project name	Nathan Phillips
31/03/2026	1.2	Finishing the end of section 3	Spencer Weishaar
06/04/2026	1.3	Complete section 6	Zema Samuel
07/04/2026	1.4	Added content to 1.2	Lincoln Earnshaw
07/04/2026	1.5	Added content to section 5	Simeon Geib
12/04/2026	1.6	Added UML Class Diagram to 4.2	Zema Samuel

©University of Kansas, 2026

Table of Contents

1. Introduction 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Definitions, Acronyms, and Abbreviations 4
   - 1.4 References 4
   - 1.5 Overview 4
2. Architectural Representation 4
3. Architectural Goals and Constraints 4
4. Logical View 6
   - 4.1 Overview 6
   - 4.2 Architecturally Significant Design Packages 6
5. Interface Description 7
6. Quality 8

 ©University of Kansas, 2026

Software Architecture Document

1. Introduction

1.1 Purpose

The SAD describes the behavior of the Algorithmic Algebraic Parser, including the tokenization and the operations of numbers observed in the command line input. The SAD also describes non-functional requirements, design constraints, and other factors in the Algorithmic Algebraic Parser to provide a complete and comprehensive description of the architecture in this project. The SAD includes an overall description of the Algorithmic Algebraic Parser and its functions, including the tokenization and completion of the numbers and operations observed in the command line.

1.2 Scope

This Software Architecture Document is a part of the overall Arithmetic Algebraic Parser project. This specific document represents the architecture of the project. This includes the formatting and layout of code, the standards to abide by, and the pipeline of the process of creating the Arithmetic Algebraic Parser.

1.3 Definitions, Acronyms, and Abbreviations

- UPEDU - The Unified Process for EDUcation is a customized, web-enabled version of the set of best practices for software engineering, using RUP (Rational Unified Process). It provides an iterative, phase-based framework to guide students through development cycles.
- SDP-PROC - Software Development Process
- SDP-PLAN - Software Development Plan

1.4 References

There currently exist no additional references outside of this document. As they are created, they will be referred to here.

1.5 Overview

Other items the SAD contains (not referenced above) include the classification of functionalities, constraints, assumptions, and dependencies, and the overall architectural description.The SAD is organized into several primary sections, starting with the introduction of the document. The introduction summarizes and introduces the purpose and context of the architectural design. Next is the overall architectural representation and goals. This section of the SAD describes technical aspects of the project, including design standards, the C++20 toolchain, and file structure constraints. The final sections cover the specific logical views and quality attributes of the project. This part outlines the modular pipeline from the Lexer to the Evaluation Engine and the non-functional requirements the project requires. At the top of the SAD document is the table of contents.

2. Architectural Representation

The architecture of the Algorithmic Algebraic Parser is represented using a subset of the Unified Process' 4+1 views. Specifically we focus on:

- 'Logical View' in section 4. This view contains a functional representation of the Algorithmic Algebraic Parser using a simple UML Class Diagram to represent the parser's modules including the Tokenizer.
- 'Implementation View' in section 3. More briefly, we also describe the implementation view of the parser in a subset of section 3. This view includes how the file structure of the source code will be organized.

3. Architectural Goals and Constraints

Build Environment

Toolchain + OS

- OS: Windows
- Compiler: LLVM
- Build System: CMake (make generator)

IDE

- **Primary:** VS Code

File Structure

- CMakeLists.txt (root)
- src/ (implementation)
- include/ (public headers)
- tests/ (unit/integration tests)
- docs/ (design notes, diagrams, Doxygen config)
- scripts/ (build helpers)
- .github/ (CI workflows, templates)

Styling & Standards

C++ Standard

- **Standard:** C++20
- **Format:** Use clang-format with committed .clang-format; run with PR.

Style

- **Headers:** #pragma once
- **Includes:** Minimal; use forward declarations
- **Namespaces:** Avoid using namespace in headers
- **Const-correctness:** Default to const where applicable
- **Ownership:** RAll policy. Use std:unique_ptr or std:shared_ptr. Raw pointers are okay when they do not own memory.
- **Errors:** Error codes and exceptions

Conventions

- Types/Classes: PascalCase
- Fonctions/variables: snake_case
- Constants: UPPER_SNAKE_CASE
- Files: snake_case.cpp/.hpp

Documentation

What must be documented

- Public classes/functions in include/
- Non-trivial algorithms, invariants, tricky edge cases
- Ownership/lifetime rules for pointers/references
- Any API that other modules call

How to write docs (recommended)

- Use Doxygen blocks on public interfaces:
    - @brief one-liner
    - @param for each parameter (include units/ranges if relevant)
    - @return description
    - @throws if exceptions are used
    - @note for relevant notes
    - @warning for relevant warnings
- Keep “why” in docs when the “how” is obvious from code

- Put examples in docs/ or as @code … @endcode
- Use per-line triple ///, rather than /** */

4. Logical View

4.1 Overview

The Algorithmic Algebraic Parser is structured around a classic compiler front-end pipeline, utilizing a modular, Object-Oriented Programming (OOP) architecture in C++. The system is decomposed into highly cohesive, loosely coupled modules that pass data sequentially to evaluate mathematical expressions.

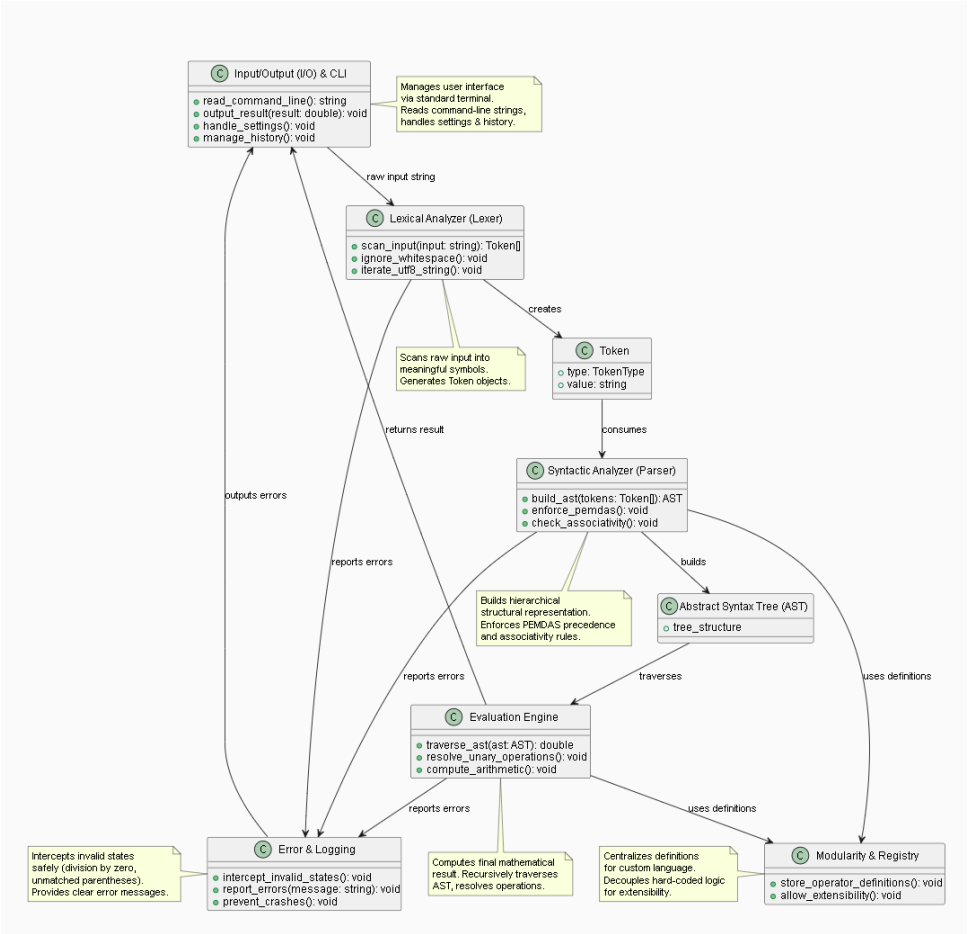
The core data flow follows a strict pipeline: SourceText → Lexer → Tokens → Parser → AST → Evaluator → Output/Logger. To support the project's goal of extensibility and the eventual addition of complex math, units, and custom symbols, the architecture also integrates an external Registry for dynamic operator and settings management, alongside a robust Error Handler to safely intercept and report invalid states across all layers.

4.2 Architecturally Significant Design Modules or Packages

Module Name	Description	Key Responsibilities
Input/Output (I/O) & CLI	Manages the user interface via a standard terminal.	Reads command-line strings, preserves whitespace, handles custom terminal features (settings, history), and outputs evaluated results clearly.
Lexical Analyzer (Lexer)	Scans raw input strings into meaningful symbols.	Iterates through UTF-8 strings, ignores extraneous whitespace, and generates sequential arrays of Token objects (e.g., numbers, operators).
Syntactic Analyzer (Parser)	Builds a hierarchical structural representation.	Consumes tokens to construct a data structure like an Abstract Syntax Tree (AST), enforcing standard PEMDAS precedence and associativity rules.
Evaluation Engine	Computes the final mathematical result.	Recursively traverses the AST, resolving unary operations and arithmetic operators against numeric constants.

 ©University of Kansas, 2026

Modularity & Registry	Centralizes definitions for the custom language.	Stores operator definitions to decouple hard-coded logic, allowing the system to embrace change and remain highly extensible for future additions.
Error & Logging	Intercepts and reports invalid system states safely.	Manages invalid scenarios (e.g., division by zero, unmatched parentheses) without crashing the program, providing clear and informative error messages to the user.



5. Interface Description

Major Interfaces

The system's primary interface is the Command Line Interface (CLI). The interaction and data flow is controlled by the **I**nput/Output (I/O) & CLI module. This module is responsible for reading from the command line and safely getting user input. See section 4.2 for more information.

Valid Inputs: All inputs must only be arithmetic operators and numeric constants that adhere to standard

mathematical rules and PEMDAS as enforced by the Syntactic Analyzer. More specifically, these inputs must be raw strings of mathematical expressions and must be imputed via the command line. White space is allowed and is handled automatically.

In the case of invalid states, like division by zero, unmatched parentheses, or similar mathematical errors, the Error & Logging module will output any relevant error messages or codes to the user without crashing the entire program. Thus, despite invalid states not being "correct" input, it will still be handled gracefully by the program.

Output Type: The output is the final mathematical result as determined by the Evaluation Engine; it is then output to the command line via the I/O module. Alternatively, as stated previously, a separate valid output is an error code or message in certain circumstances.

6. Quality

Reliability - Reliability is achieved through strict input validation and deterministic parsing behavior. Invalid or malformed inputs are caught early in tokenization and handled gracefully using structured error reporting, ensuring the system does not crash or exhibit undefined behavior. Testing ensures correctness across normal and edge cases.

Maintainability - The system uses a modular design that separates tokenization, parsing, and related logic. This reduces coupling and makes the codebase easier to modify or debug. Consistent coding standards and documentation further improve readability and maintainability.

Portability - The parser is implemented in standard C++20 and uses CMake for cross-platform builds. It does not depend on platform-specific libraries, allowing it to run on Windows, Linux, and macOS with minimal changes.

Extensibility - The architecture supports the addition of new operators and grammar rules with minimal changes to existing components. Clear separation of responsibilities allows new features to be integrated without redesigning the system. This enables future expansion of functionality.

Performance - The parser operates in approximately linear time relative to input size, as tokenization and parsing are performed in a single pass. Efficient STL structures and minimal memory allocation reduce overhead. This allows the system to handle larger expressions without significant performance loss.

Security, Safety, and Privacy - The system treats all input strictly as data and does not execute arbitrary code, reducing security risks. Malformed or adversarial input is handled safely through validation and controlled error handling. Safety is not a concern as the system does not interact with physical systems, and privacy concerns are minimal since no user data is stored or transmitted.